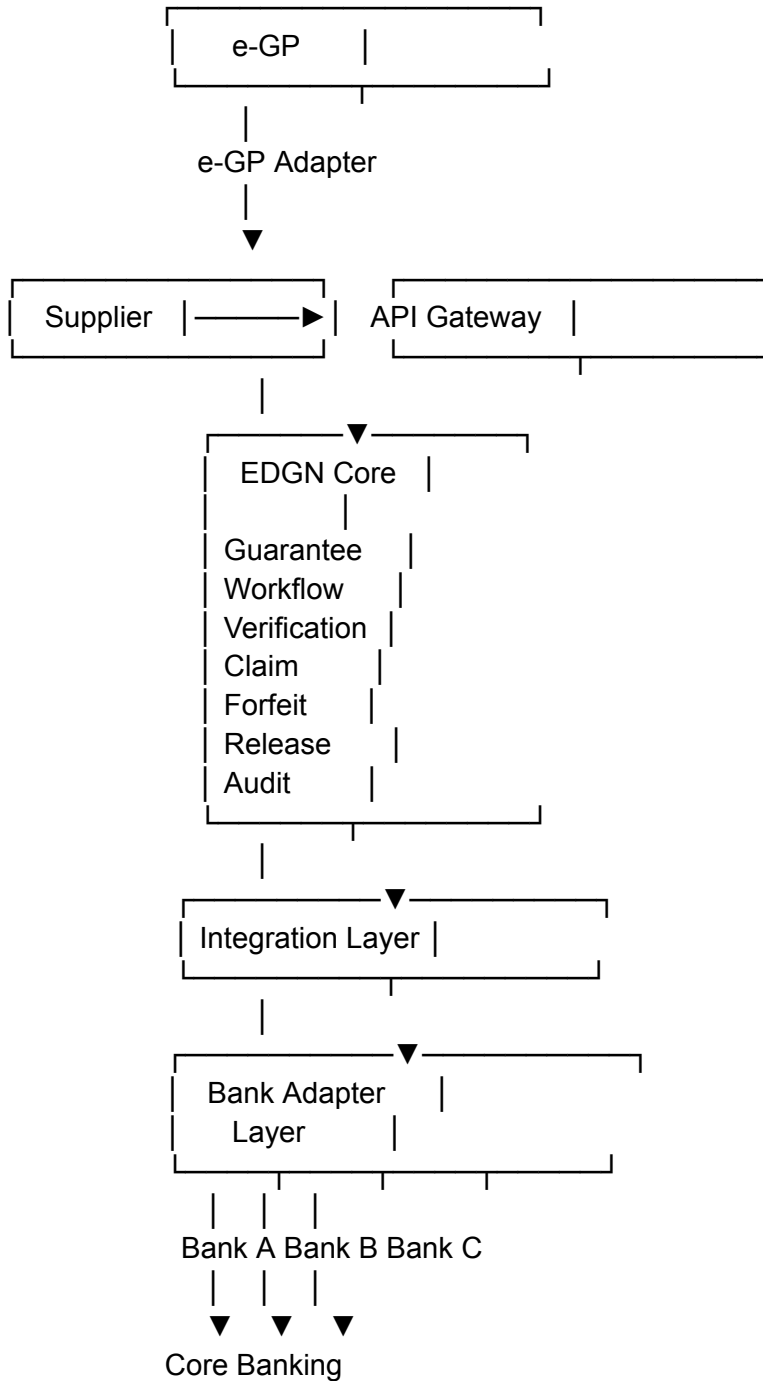


እሺ። Developer እንዲገነባው **architecture** ብቻ ከፈለግህ፣ ይህን በቀጥታ ላክለት፦

EDGN – Developer Architecture Requirement

EDGNን **API-First, Multi-Bank Digital Guarantee Infrastructure** እንድትገነባልኝ እፈልጋለሁ።

Architectureው እንዲህ ይሆን፦



1. API Gateway

All external requests must pass through the API Gateway.

It must handle:

- Authentication
- Authorization
- Rate limiting
- Request validation
- API versioning
- Correlation ID
- Logging
- Threat protection

Use:

`/api/v1/...`

2. EDGN Core

EDGN Core must contain the business logic, NOT bank-specific logic.

Core modules:

- Guarantee Management
- Guarantee Workflow Engine
- Tender Management
- Customer/Supplier Management
- Bank Management
- Collateral Management
- Digital Signature
- Verification
- Claim Management
- Forfeiture Management
- Release Management
- Notification
- Audit Trail
- Reporting

3. Bank Adapter Layer

This is mandatory.

Create a standard interface:

BankGuaranteeAdapter

verifyCustomer()
verifyAccount()
checkEligibility()
checkLimit()
checkCollateral()
reserveCollateral()

createGuarantee()
approveGuarantee()
issueGuarantee()

amendGuarantee()
extendGuarantee()
cancelGuarantee()

claimGuarantee()
forfeitGuarantee()
releaseGuarantee()

getGuaranteeStatus()

Then implement:

CBEAdapter
AwashAdapter
BOAAdapter
DashenAdapter
OtherBankAdapter

Adding a new bank must only require a new adapter.

EDGN Core must NOT be modified because a new bank is added.

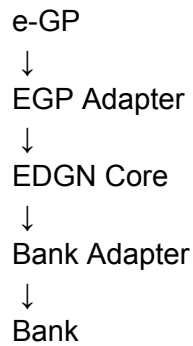
4. e-GP Adapter

Create a separate:

EGPAdapter

It must isolate all e-GP-specific API logic from EDGN Core.

Architecture:

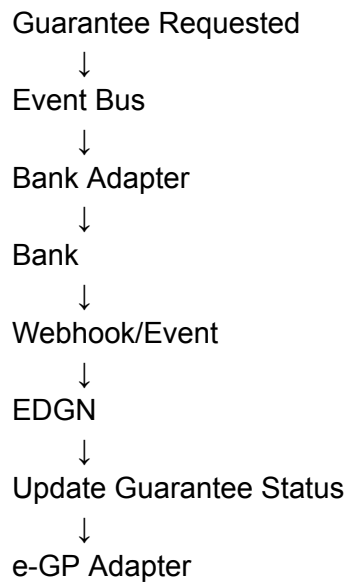


Do not hard-code assumed e-GP endpoints. Implement the adapter according to the official e-GP API specification when provided.

5. Event-Driven Integration

Use an event/message layer such as Kafka or RabbitMQ.

Example:



6. Database

Use PostgreSQL.

Core entities should include:

users
suppliers
procuring_entities
banks
bank_branches
tenders
guarantees
guarantee_parties
guarantee_documents
collaterals
claims
forfeitures
releases
digital_signatures
external_references
integration_transactions
webhook_events
audit_logs
notifications

7. Security Architecture

Implement:

- OAuth2/OIDC
- mTLS for bank integrations where required
- RBAC
- Encryption
- Secure secrets management
- Digital signatures
- API rate limiting
- Idempotency
- Replay protection
- Immutable audit logs
- HSM/key-management integration where required

8. Guarantee State Machine

Implement strict lifecycle control:

DRAFT

↓

SUBMITTED

↓

BANK_REVIEW

↓
 COLLATERAL_VERIFICATION
 ↓
 APPROVED
 ↓
 DIGITALLY_SIGNED
 ↓
 ISSUED
 ↓
 EGP_SUBMITTED
 ↓
 ACTIVE
 ↓
 AMENDED / EXTENDED
 ↓
 CLAIMED / FORFEITED / RELEASED

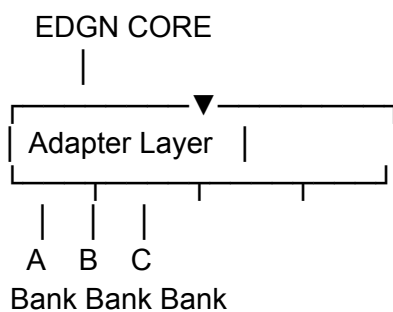
Invalid state transitions must be rejected.

9. Important Architecture Principle

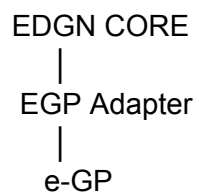
Do NOT build:

EDGN → Bank A API
 EDGN → Bank B API
 EDGN → Bank C API

Build:



And for e-GP:



Final Requirement

The architecture must be **bank-neutral, e-GP-independent, API-first, event-driven, secure, scalable, and extensible**.

The objective is that EDGN can connect to **multiple banks and e-GP** without changing the **EDGN Core**.